Scaling Redis is best understood as **scaling along four independent dimensions**:

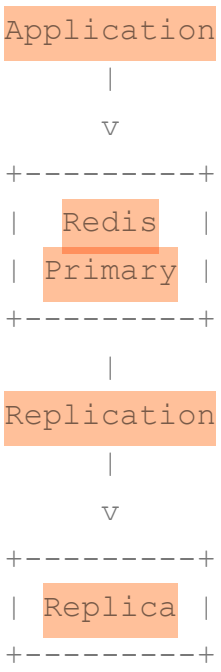
1. **Vertical scaling** — make one Redis node larger/faster.
2. **Replication + read scaling** — add replicas and distribute reads.
3. **Partitioning/sharding** — distribute keys across multiple Redis nodes.
4. **Geographic scaling** — distribute Redis deployments across regions.

For a MAANG+ system-design interview, the important part is not merely knowing Redis Cluster. You should be able to explain **when to use each mechanism, how requests are routed, what happens during node failure, and how the system behaves while scaling up/down**.

---

1. Start with the baseline

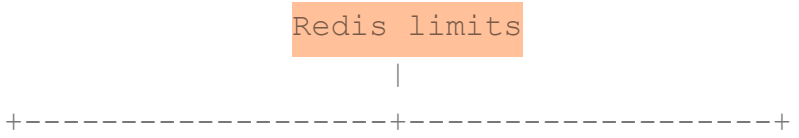
A simple Redis deployment looks like:

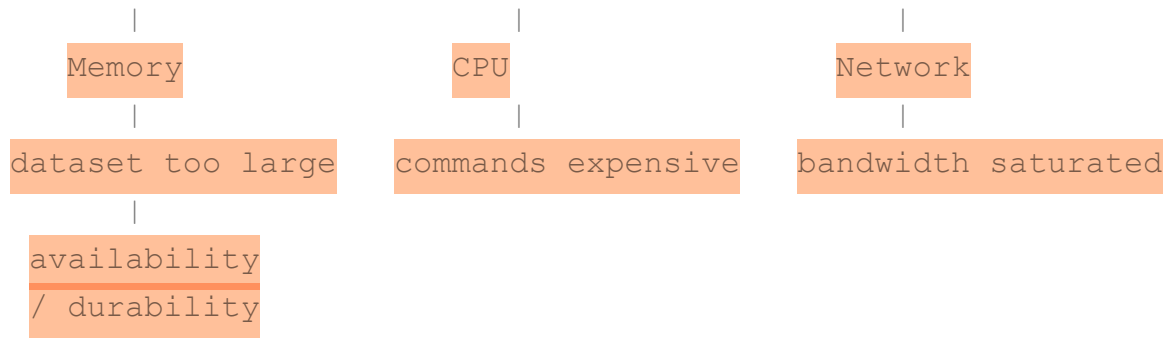


Initially, this can be surprisingly powerful.

Redis is primarily an **in-memory data store**, so a single sufficiently large machine can handle substantial throughput with very low latency.

But eventually one or more limits become bottlenecks:



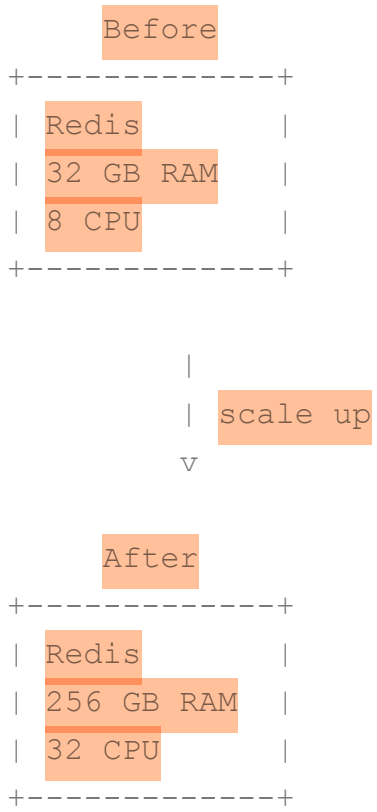


The scaling strategy depends on **which resource is actually saturated**.

---

2. Vertical scaling

The simplest approach is:



Increase:

- RAM
- CPU
- network bandwidth
- potentially faster storage for persistence

Advantages

- Extremely simple.
- No application-level partitioning.
- No cross-node coordination.
- Existing Redis commands continue to work.
- No change to key distribution.

Limitations

There is a physical ceiling.

More importantly, a single Redis primary remains a **single failure domain**.

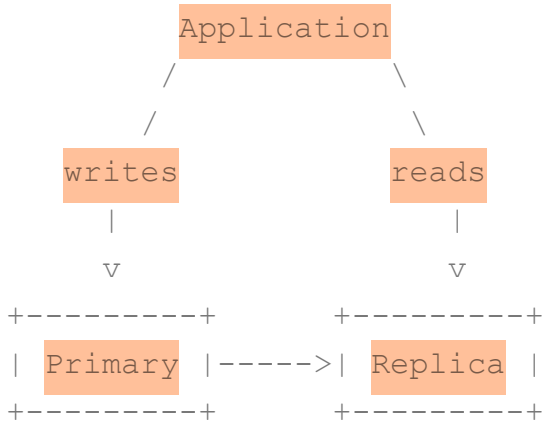
Therefore:

> Vertical scaling increases capacity, but does not fundamentally solve availability or horizontal scalability.

---

3. Add replicas for availability and read scaling

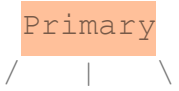
The next step is replication.



Redis replication is generally **asynchronous**.

The primary processes writes and propagates them to replicas.

You can add multiple replicas:



```
          /     |     \
         v      v      v
Replica Replica Replica
```

This provides two important capabilities.

Availability

If the primary fails:

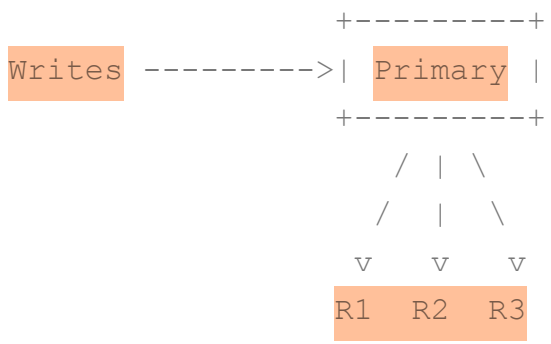
```
Primary
   X

Replica A  ---> promoted
Replica B
```

A failover mechanism can promote a replica.

Read scaling

Read traffic can be distributed:



However, there is an important interview caveat:

Replicas are eventually consistent with the primary

Suppose:

```
t1: SET user:1 balance 100
t2: immediately GET user:1
```

If the GET goes to a replica before replication catches up, it might return the previous value.

Therefore:

> Redis replicas are excellent for read scaling, but you must explicitly consider replica lag and read-after-write consistency.

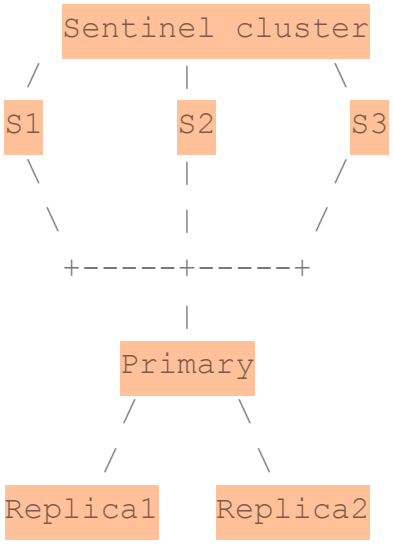
For workloads requiring strong read-after-write behavior, route those reads to the primary or use an appropriate consistency strategy.

---

4. Redis Sentinel

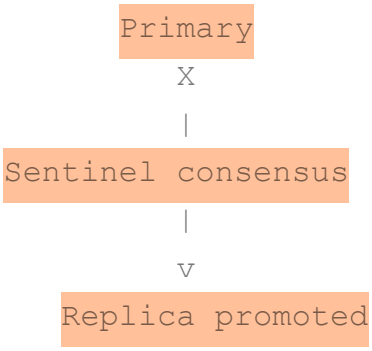
For a non-clustered Redis deployment, **Redis Sentinel** can provide monitoring and automatic failover.

A typical topology:



Sentinels monitor Redis instances.

If the primary fails:



Sentinel is primarily about:

- monitoring

- failure detection
- leader/primary election
- failover
- service discovery

It is **not a horizontal data-partitioning solution**.

This distinction is important:

Mechanism	Main purpose
Vertical scaling	Capacity
Replication	Availability + reads
Sentinel	Automatic failover
Redis Cluster	Horizontal partitioning + failover

---

5. When replication is no longer enough: sharding

Suppose your dataset is:

```
1 TB
```

but your largest practical Redis server can only hold:

```
256 GB
```

Replication doesn't solve this.

If you create:

```
Primary: 256 GB
Replica: 256 GB
Replica: 256 GB
```

you still have only **256 GB of unique data capacity**.

The replicas contain copies.

You need **partitioning**.

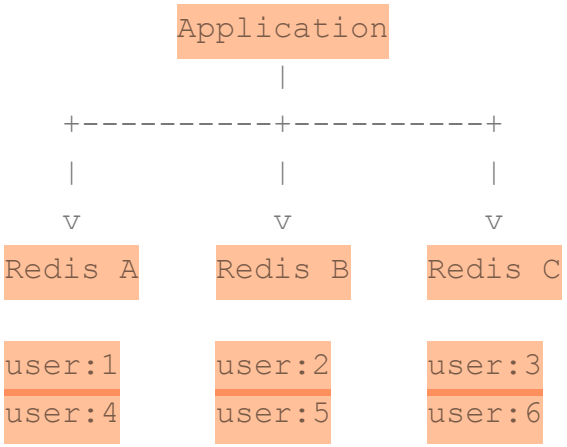
---

6. Redis sharding

Instead of putting every key on one Redis server:

```
Redis
  |
  +-- user:1
  +-- user:2
  +-- user:3
  +-- ...
```

partition keys:



Now total capacity is approximately:

```
Total capacity ≈
capacity(A) + capacity(B) + capacity(C)
```

and aggregate throughput can increase as well.

---

7. How does Redis decide which shard owns a key?

This is where **consistent hashing / hash partitioning** concepts become important.

A simple model is:

```
shard = hash(key) % N
```

For example:

```
hash("user:101") % 4 = 2
```

Therefore:

```
user:101 → shard 2
```

However, naïve modulo hashing creates a major scaling problem.

Suppose:

```
N = 4
```

and you increase it to:

```
N = 5
```

Almost every key can map to a different shard.

That means massive data movement.

Redis Cluster uses a different mechanism.

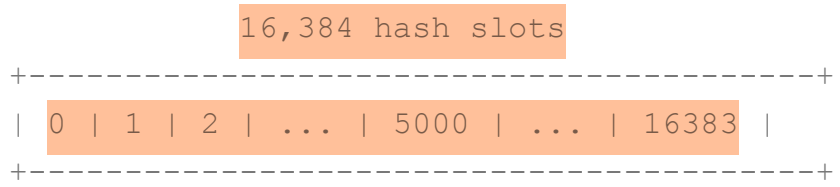
---

8. Redis Cluster and hash slots

Redis Cluster divides the keyspace into:

16,384 hash slots

Conceptually:



A key is mapped to a slot:

```
slot = CRC16(key) % 16384
```

Then slots are assigned to cluster nodes.

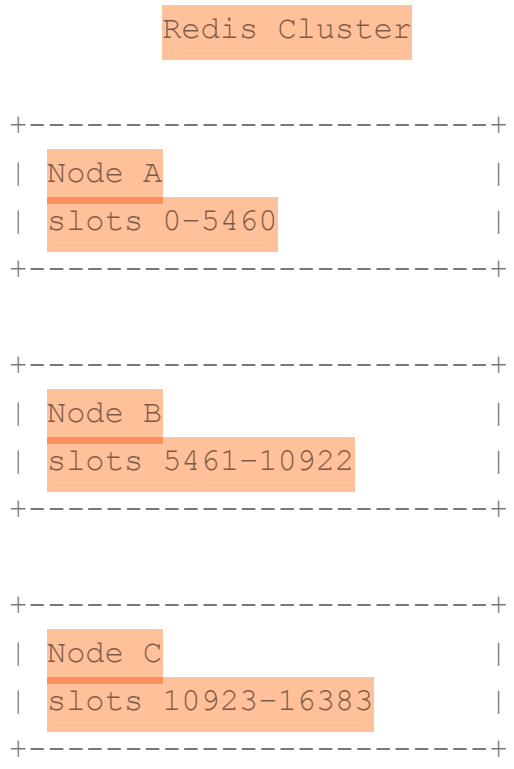
For example:

```
Node A
  slots 0 - 5460

Node B
  slots 5461 - 10922

Node C
  slots 10923 - 16383
```

Conceptually:



This indirection is extremely useful.

Instead of saying:

```
key → physical server
```

we have:

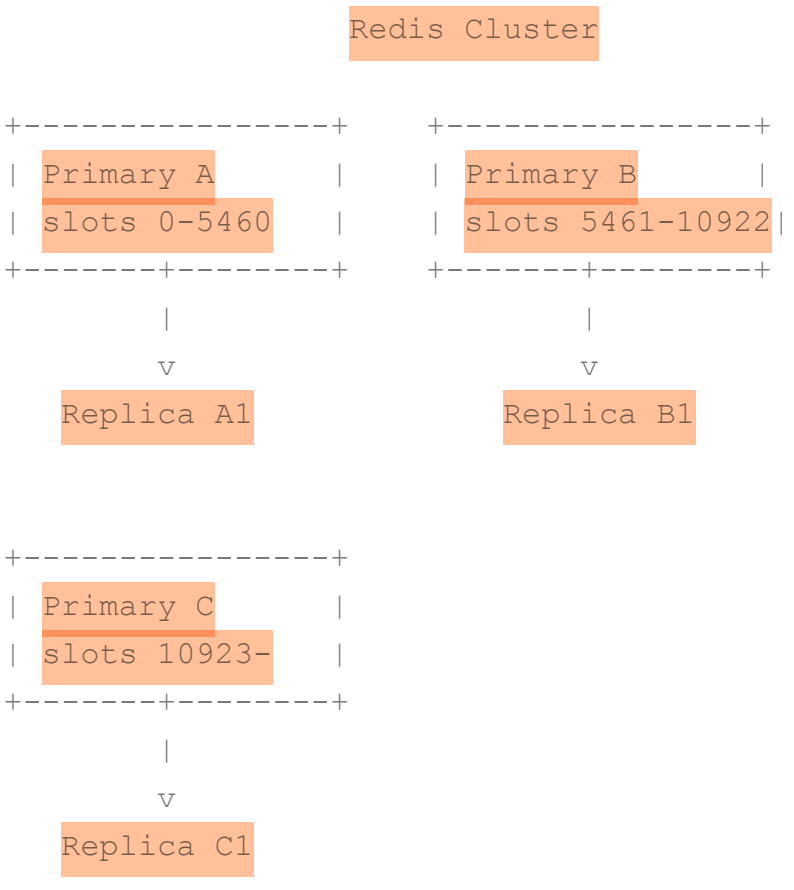
```
key
 ↓
hash slot
 ↓
node
```

This makes **resharding** much easier.

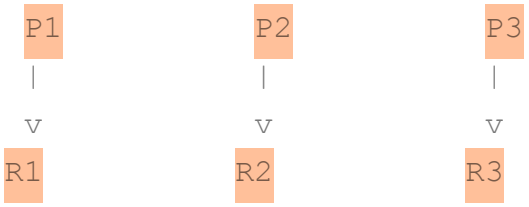
---

9. Redis Cluster with replicas

A production cluster usually looks more like:



Or:



Each primary owns a subset of hash slots.

Each primary has one or more replicas.

Now we get:

- horizontal capacity
- horizontal throughput
- high availability
- replica-based failover

---

10. How a Redis Cluster client works

A cluster-aware Redis client maintains information about:

```
slot → node
```

For example:

```
slot 100 → P1
slot 6000 → P2
slot 12000 → P3
```

The application computes:

```
hash(key) → slot
```

and sends the command directly to the appropriate primary.

```
Application
      |
      | SET user:123 ...
      |
      v
```

```
hash("user:123")
      |
      v
slot 7421
      |
      v
Primary B
```

This avoids a centralized Redis proxy becoming a bottleneck.

---

11. MOVED redirection

The client's slot map may become stale.

Suppose the client believes:

```
slot 7421 → Node B
```

but the cluster has moved that slot to Node C.

The client sends:

```
GET user:123
```

to Node B.

Node B responds conceptually:

```
MOVED 7421 Node C
```

The client then:

1. updates its slot map
2. retries against Node C

```
Client
  |
  | GET user:123
  v
Node B
```

```
    |
    | MOVED 7421 → Node C
    v
Client updates routing
    |
    v
Node C
```

This is one of the important details to understand about Redis Cluster.

---

12. Scaling out

Suppose you currently have:

```
P1
P2
P3
```

and need more capacity.

Add:

```
P4
```

Initially:

```
P1 —— slots A
P2 —— slots B
P3 —— slots C
P4 —— no/limited slots
```

You then **reshard**.

Move some slots:

```
Before:

P1 → 5,461 slots
P2 → 5,461 slots
```

```
P3 → 5,462 slots
```

```
After:
```

```
P1 → ~4,096
P2 → ~4,096
P3 → ~4,096
P4 → ~4,096
```

The important point:

> You move slots, not the entire dataset blindly.

Keys belonging to those slots move with them.

---

13. What happens during resharding?

Consider:

```
P1
 |
 | slots 0–5460
```

We decide:

```
slot 3000 → P4
```

Data associated with slot 3000 needs to migrate:

```
              migration
P1 -------------------------> P4
        keys in slot 3000
```

During migration, Redis Cluster has mechanisms to coordinate access to migrating/importing slots.

A cluster-aware client may also receive temporary redirections such as:

```
ASK
```

This is different from `MOVED`.

`MOVED`

Means:

> This slot permanently belongs somewhere else; update your routing table.

`ASK`

Means:

> This particular request should temporarily be sent to the target during slot migration.

That distinction is excellent interview material.

---

14. Scaling down

Scaling down is essentially the reverse.

Suppose:

```
P1
P2
P3
P4
```

and P4 is underutilized.

You first migrate its slots:

```
P4
 |
 +----> P1
 +----> P2
 +----> P3
```

Once P4 owns no slots:

```
P1
P2
P3
```

you can remove P4.

Important principle

Never think:

> "Terminate Redis node P4."

Think:

> "Drain its hash slots, migrate the data, verify ownership, then remove the node."

This is analogous to draining a server before terminating it in a distributed system.

---

15. The hardest scaling problem: hot keys

Sharding solves **dataset size** and aggregate throughput, but it can introduce a major problem:

Hot keys

Suppose:

```
user:123
```

is accessed:

```
500,000 requests/sec
```

while every other key receives only:

```
100 requests/sec
```

Hashing doesn't help.

That key still maps to one slot:

```
user:123
   |
   v
slot 7421
   |
   v
Primary B
   |
   v
500K requests/sec
```

So:

> Sharding distributes keys, not necessarily traffic.

---

16. Hot-key mitigation

One common approach is **key replication / key splitting**.

Instead of:

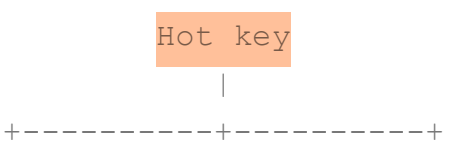
```
user:123
```

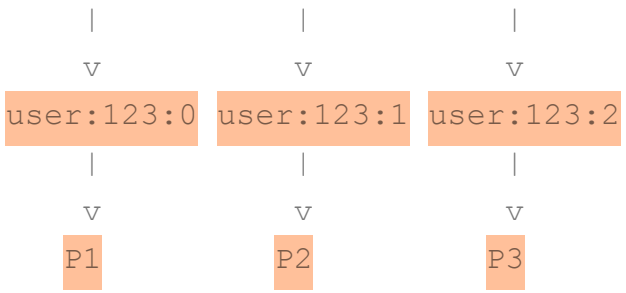
use:

```
user:123:0
user:123:1
user:123:2
...
user:123:N
```

Requests are distributed across replicas/shards.

For example:





For counters, you might use **striped/sharded counters**:

```
counter:0
counter:1
counter:2
...
counter:31
```

Then:

```
total = Σ counter[i]
```

This trades some read/aggregation complexity for much higher write scalability.

---

17. Multi-key operations are a major Redis Cluster constraint

This is extremely important.

Suppose you execute:

```
MGET user:1 user:2 user:3
```

If these keys are distributed across different slots:

```
user:1 → P1
user:2 → P2
user:3 → P3
```

Redis Cluster cannot treat this as a normal single-node atomic operation.

This affects:

- `MGET`
- `MSET`
- transactions
- Lua scripts
- certain multi-key commands

Therefore:

> Your data model must be designed around the sharding key.

---

18. Hash tags

Redis Cluster provides **hash tags** to intentionally place related keys in the same slot.

For example:

```
user:{123}:profile
user:{123}:orders
user:{123}:cart
```

The `{123}` portion is used for hashing.

Therefore:

```
user:{123}:profile ┐
user:{123}:orders  ┼──> same slot
user:{123}:cart    ┘
```

Now multi-key operations involving these keys can be executed on the same node.

This is a very important data-modeling technique.

But don't overuse it.

If you put too many keys under the same hash tag:

```
{global}:key1
{global}:key2
```

```
{global}:key3
...
```

you can accidentally create a **hot slot**.

---

19. Redis Cluster doesn't magically solve consistency

Redis Cluster generally prioritizes availability and performance rather than providing strong distributed consistency across all nodes.

A typical write path is:

```
Client
   |
   v
Primary
   |
   | asynchronous replication
   +----------> Replica 1
   |
   +----------> Replica 2
```

There can therefore be a window:

```
Primary accepted write
        |
        | replication delay
        v
Replica doesn't have write yet
```

If the primary fails before replication reaches a replica, that write may be lost during failover.

For systems where losing a recently acknowledged write is unacceptable, you need to carefully evaluate:

- replication acknowledgements
- `WAIT`
- persistence configuration

- failover semantics
- application-level durability guarantees

Do not claim Redis replication is equivalent to synchronous consensus replication.

---

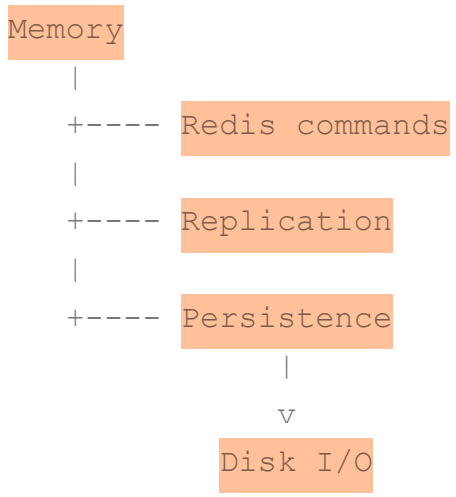
20. Persistence and scaling

Redis can be configured with persistence mechanisms such as:

- RDB snapshots
- AOF
- combinations of persistence approaches

Persistence creates another dimension of scalability.

For example:



Large datasets can make:

- snapshotting expensive
- AOF rewrite expensive
- recovery slower
- restart time significant

Therefore, when designing a production Redis system, ask:

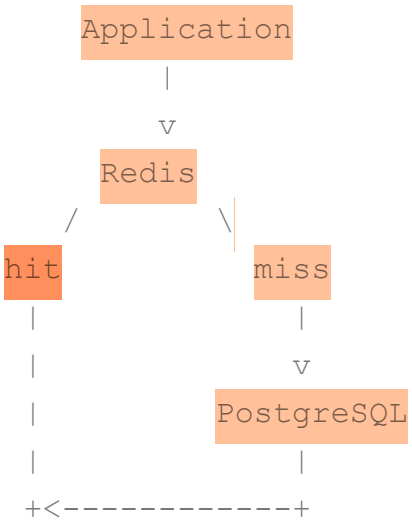
> Is Redis the authoritative datastore, or is it a cache?

That dramatically changes the durability requirements.

---

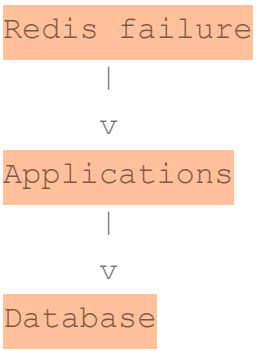
21. Redis as a cache

A common architecture is:



Redis contains derived/cached data.

If Redis completely disappears:



the database remains authoritative.

This means Redis scaling/failure strategies can prioritize:

- latency
- availability
- throughput

over absolute durability.

---

22. But watch for cache stampede

Suppose a popular cache entry expires:

```
product:123
```

10,000 requests simultaneously miss:

```
10,000 requests
      |
      v
    Redis
      |
      | miss
      v
 PostgreSQL
```

Now the database gets hammered.

Mitigations include:

- TTL jitter
- request coalescing
- distributed locks
- stale-while-revalidate
- background refresh
- probabilistic early expiration

For example:

```
Request 1 ─┐
Request 2 ─┤
Request 3 ─┼──> one DB request
Request 4 ─┤
Request 5 ─┘
```

and the result is populated back into Redis.

---

23. Memory management becomes critical

Redis is memory-centric.

You need to monitor:

```
used_memory
used_memory_rss
maxmemory
fragmentation
evictions
```

Set an appropriate `maxmemory` policy depending on the workload.

Common eviction-policy concepts include:

```
noeviction
allkeys-lru
allkeys-lfu
volatile-lru
volatile-lfu
TTL-based policies
```

The choice depends on whether:

- all keys are cacheable
- only TTL keys may be evicted
- recency matters
- frequency matters
- writes must fail instead of evicting data

For a cache, an eviction policy such as LFU can be attractive when access frequency is a better predictor of future popularity than recency.

---

24. Memory fragmentation

Redis memory usage isn't necessarily equivalent to your logical dataset size.

For example:

```
Logical data:
100 GB
```

```
Redis process RSS:
130 GB
```

because of allocator fragmentation and other overhead.

This means capacity planning must account for:

- key overhead
- value overhead
- data structure overhead
- allocator fragmentation
- replication buffers
- client buffers
- persistence overhead

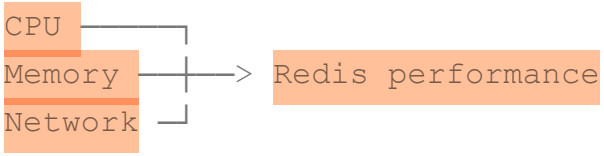
Don't simply say:

> "My dataset is 100 GB, therefore I need a 100 GB Redis server."

---

25. Network becomes a bottleneck

At sufficiently high throughput:



becomes important.

Consider:

```
1 million requests/sec
×
average response = 1 KB
```

That's approximately:

```
1 GB/sec
```

of payload traffic before protocol overhead and request traffic.

Therefore, scaling Redis isn't only about RAM and CPU.

You need to consider:

- network bandwidth
- packet rate
- request/response sizes
- pipelining
- batching
- connection count

---

26. Pipelining

If the application performs:

```
GET A
GET B
GET C
GET D
```

with four round trips:

```
Client → Redis
       ← Redis

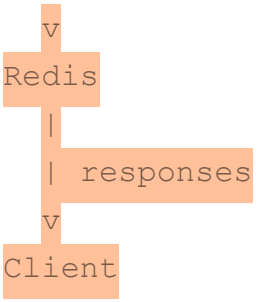
Client → Redis
       ← Redis

...
```

latency increases.

Pipelining allows:

```
Client
  |
  | GET A
  | GET B
  | GET C
  | GET D
```



This reduces network round trips and can dramatically increase throughput.

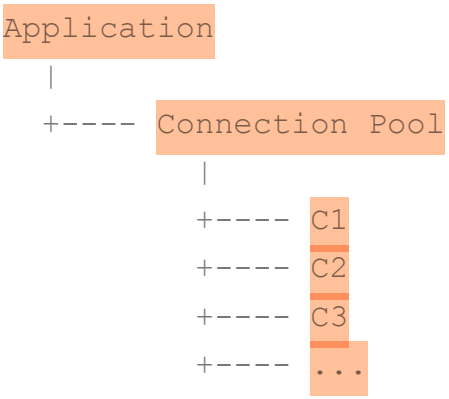
But pipelining does **not** magically make commands atomic.

---

27. Connection management

Don't create a Redis connection per request.

Instead:



Too many connections cause:

- memory overhead
- CPU overhead
- context switching
- connection management overhead

Too few cause:

- request queueing
- increased latency

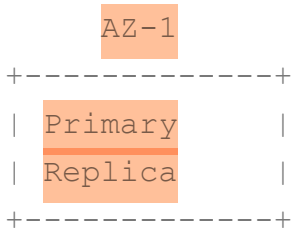
Connection pool sizing should therefore be load-tested rather than arbitrarily increased.

---

28. Multi-AZ architecture

For production availability, don't place primary and replica in the same failure domain.

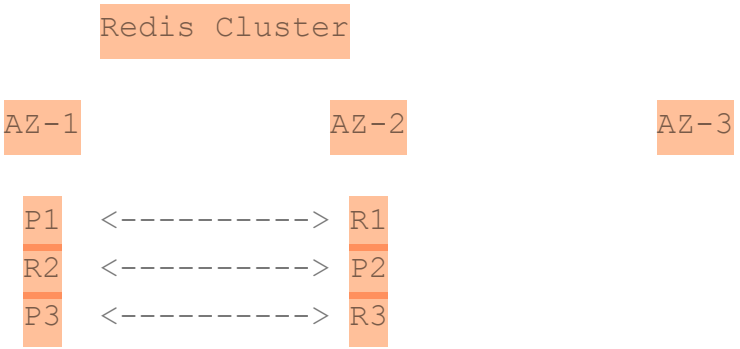
Bad:



Better:



For a cluster:



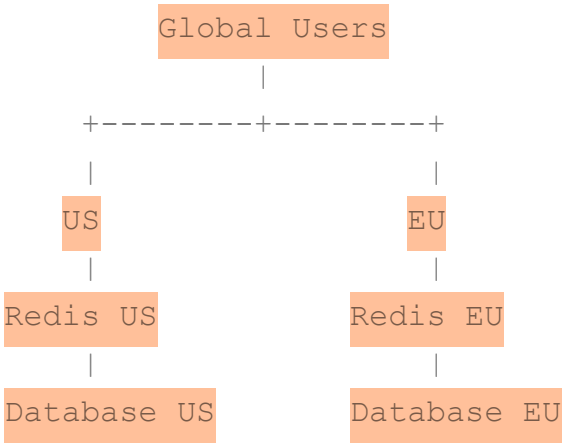
You want replicas positioned so that loss of an AZ does not eliminate both the primary and its failover candidate.

---

29. Multi-region Redis

This is substantially more complicated.

You could have:

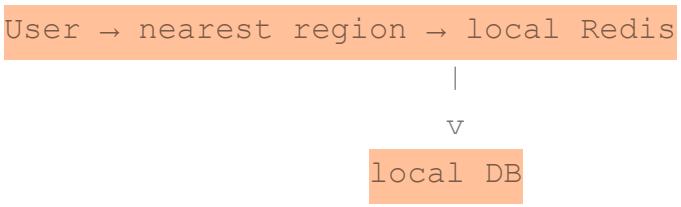


The major question is:

> Is Redis globally authoritative, or is each region using Redis as a local cache?

Local cache

Usually easier:



Each region can independently rebuild its cache.

Global mutable state

Much harder.

Examples:

- global counters
- inventory
- distributed locks
- globally unique state
- financial balances

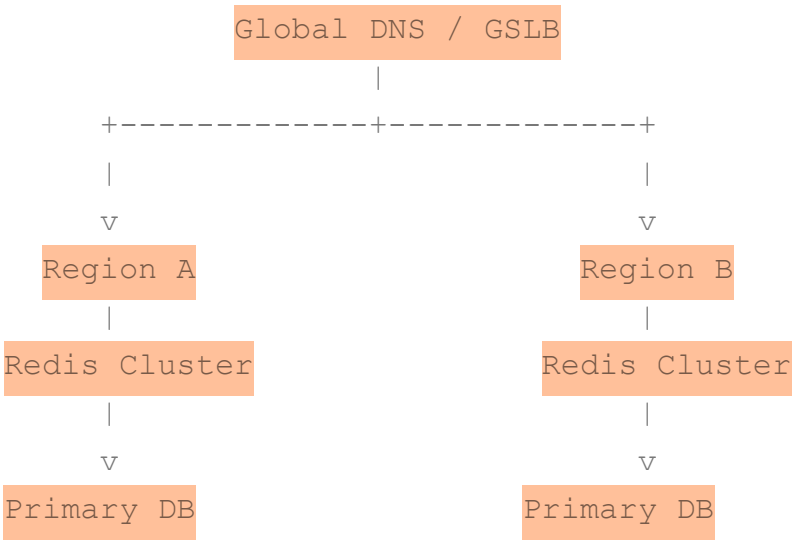
Cross-region consistency and failure handling become major design concerns.

For these workloads, Redis should generally not be treated as a magic global consistency layer.

---

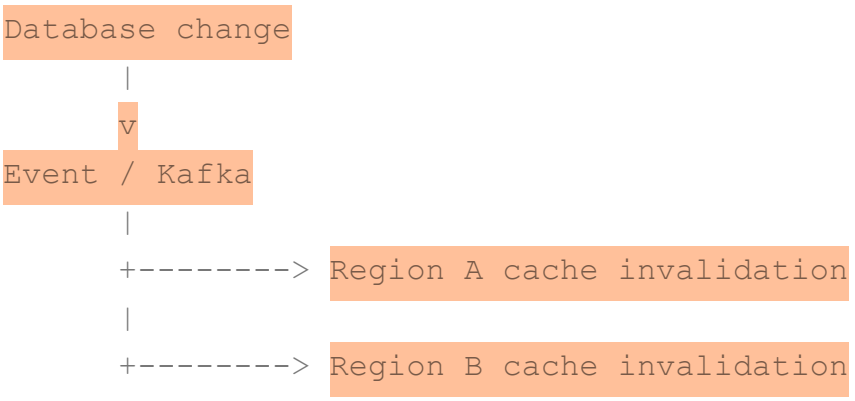
30. Global caching pattern

For a globally distributed application, a common pattern is:



Each region has its own Redis cluster.

Cache invalidation can happen through:



This is often more operationally robust than trying to create one giant globally shared Redis cluster.

---

31. Failure scenario: Redis primary dies

Consider:



```
+---- R1
+---- R2
```

P1 fails:

```
P1 X

R1
R2
```

Failover mechanism detects the failure and promotes a replica:

```
      R1
      |
      v
new Primary
```

Clients need to discover the new topology.

During failover you should expect:

- transient errors
- retries
- some increased latency
- possible loss of unreplicated writes

Therefore applications need:

- retry policies
- timeouts
- idempotent operations where possible
- circuit breakers where appropriate

---

32. Failure scenario: entire shard dies

Suppose:

```
P2
 |
```

```
R2
```

and P2 dies.

If R2 is healthy:

```
P2 X
 |
R2 → promoted
```

The slot ownership can continue through the promoted node.

But if both primary and replica are lost:

```
P2 X
R2 X
```

then the shard's data may become unavailable or lost depending on persistence/recovery architecture.

This is why replica placement and backup/recovery strategy matter.

---

33. Failure scenario: database survives but Redis disappears

For a cache:

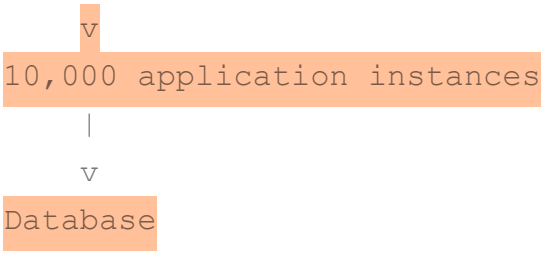
```
Redis X

      |
      v
Database
```

You can rebuild Redis by gradually warming it.

But beware:

```
Redis restart
     |
```



This can create a **thundering herd**.

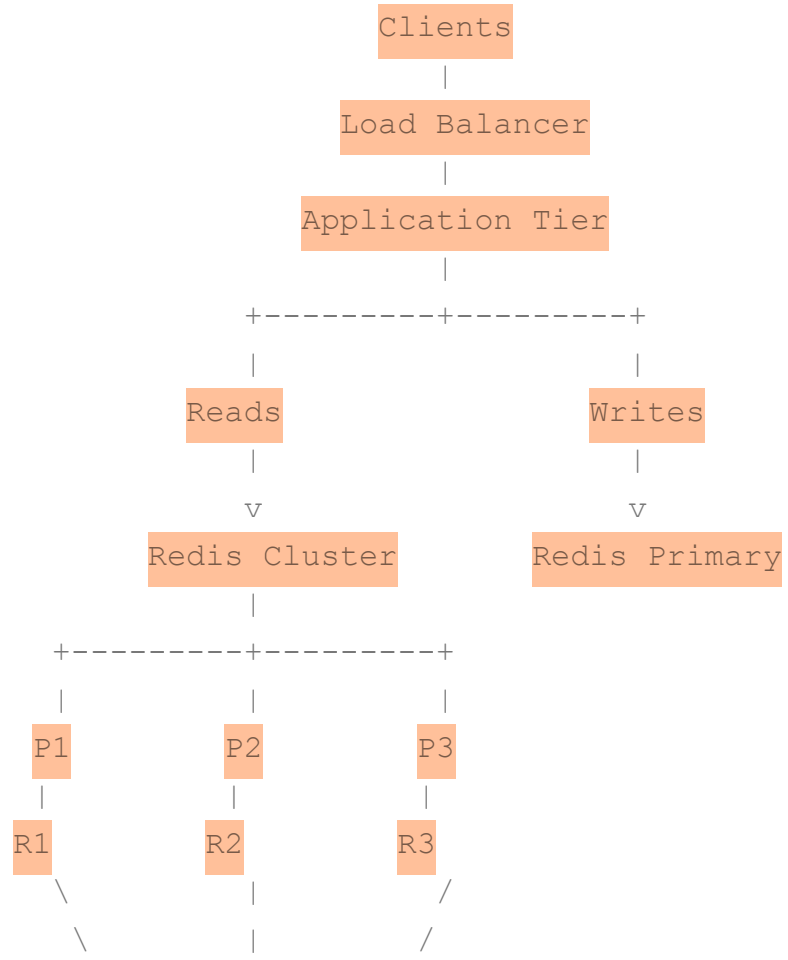
Solutions include:

- controlled cache warming
- request rate limiting
- randomized refresh
- backpressure
- database protection
- gradual traffic ramp-up

---

34. A useful production architecture

For a large application, a representative architecture is:



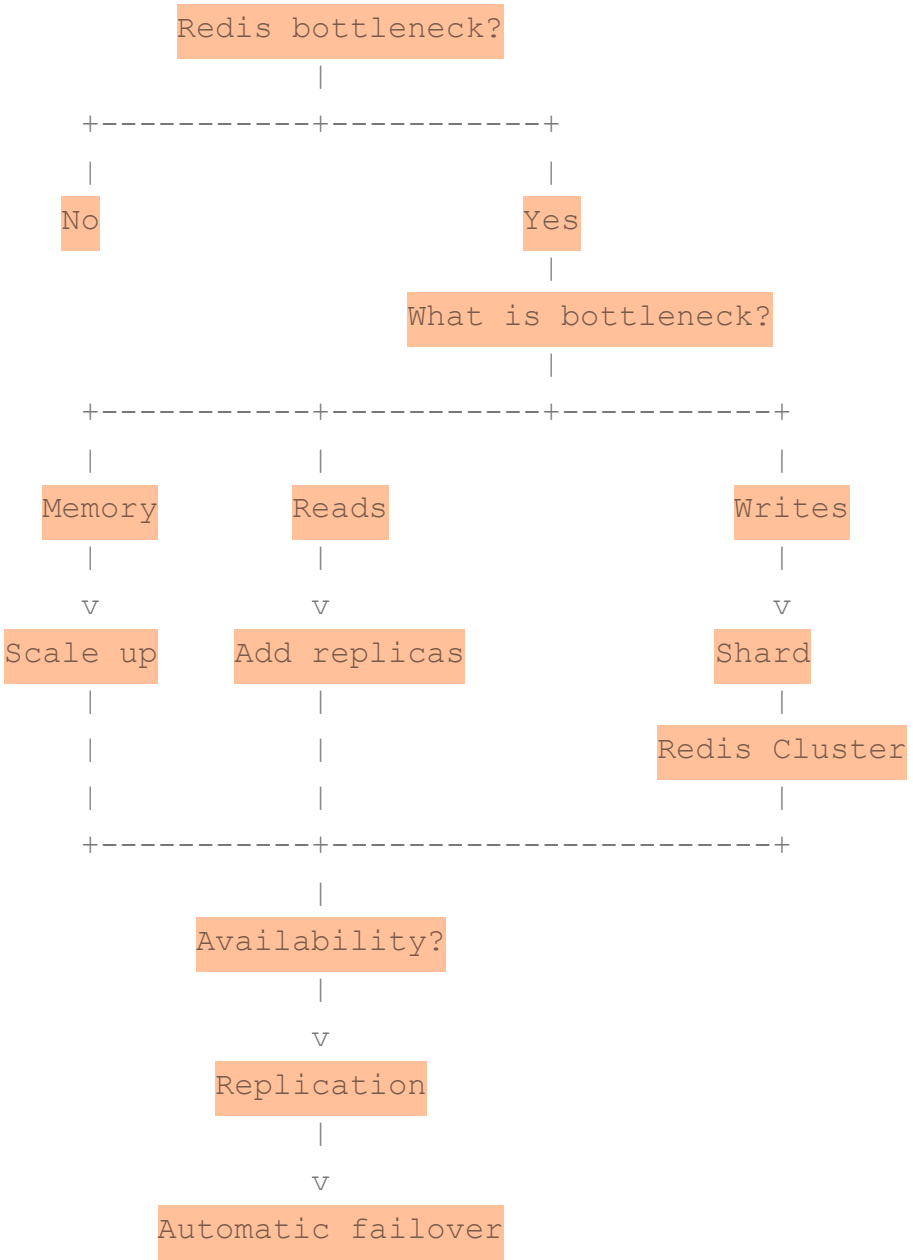


But the exact architecture depends heavily on workload.

---

35. Scaling decision tree

For interviews, I recommend thinking about Redis scaling in this sequence:



Then investigate:

```
Hot keys?
Cross-key operations?
Multi-region?
Persistence?
Network?
Connection count?
Eviction?
Cache stampede?
```

---

36. Scaling Redis: what each technique solves

Technique	Dataset capacity	Read throughput	Write throughput	HA
Vertical scaling	↑	↑	↑	❌
Replica	–	↑↑	–	↑
Sentinel	–	–	–	↑↑
Redis Cluster	↑↑	↑↑	↑↑	↑
More replicas/shard	–	↑↑	–	↑
Multi-region	↑	↑	depends	↑↑
Hot-key splitting	–	↑	↑↑	depends

The key distinction is:

> **Replication duplicates data; sharding divides data.**

That single sentence is worth remembering for interviews.

---

37. A concrete scaling journey

Imagine an e-commerce system.

Initially:

```
Application
     |
   Redis
```

Traffic:

```
20K ops/sec
dataset = 10 GB
```

A single Redis instance is sufficient.

Stage 1 — availability

Add:

```
Primary
   |
Replica
```

Now:

```
20K ops/sec
10 GB dataset
HA required
```

Stage 2 — read scaling

Traffic becomes:

```
100K reads/sec
20K writes/sec
```

Add replicas:

```
          Primary
        /    |    \
      R1    R2    R3
```

Distribute reads.

Stage 3 — memory limit

Dataset becomes:

```
500 GB
```

A single primary cannot comfortably accommodate it.

Move to:

```
P1   P2   P3   P4
```

with Redis Cluster.

Stage 4 — availability

Add replicas:

```
P1 → R1
P2 → R2
P3 → R3
P4 → R4
```

Now each shard has a failover candidate.

Stage 5 — scale out

Dataset becomes:

```
2 TB
```

Add:

```
P5
P6
```

```
P7
P8
```

Redis Cluster rebalances hash slots.

Stage 6 — hot key

A celebrity/product becomes extremely popular:

```
product:999
```

and one shard becomes overloaded.

Sharding doesn't solve it.

Introduce:

```
product:999:0
product:999:1
product:999:2
...
```

or another workload-specific hot-key strategy.

---

38. The most important interview concepts

For **MAANG+ system design**, I would make sure you can explain these deeply:

Core

1. Redis data structures
2. In-memory architecture
3. persistence: RDB/AOF
4. TTL and expiration
5. eviction policies
6. memory fragmentation

Availability

7. primary/replica architecture
8. asynchronous replication
9. Sentinel
10. automatic failover
11. replica lag
12. split-brain/failure detection considerations

Horizontal scaling

13. Redis Cluster
14. hash slots
15. key → slot → node routing
16. `MOVED`
17. `ASK`
18. resharding
19. node addition/removal
20. scaling up/down

Data modeling

21. hash tags
22. multi-key commands
23. transactions across shards
24. Lua/scripts and cluster constraints
25. hot keys
26. key distribution

Performance

27. pipelining
28. batching
29. connection pooling
30. network bottlenecks
31. large values
32. expensive commands
33. latency vs throughput

Production

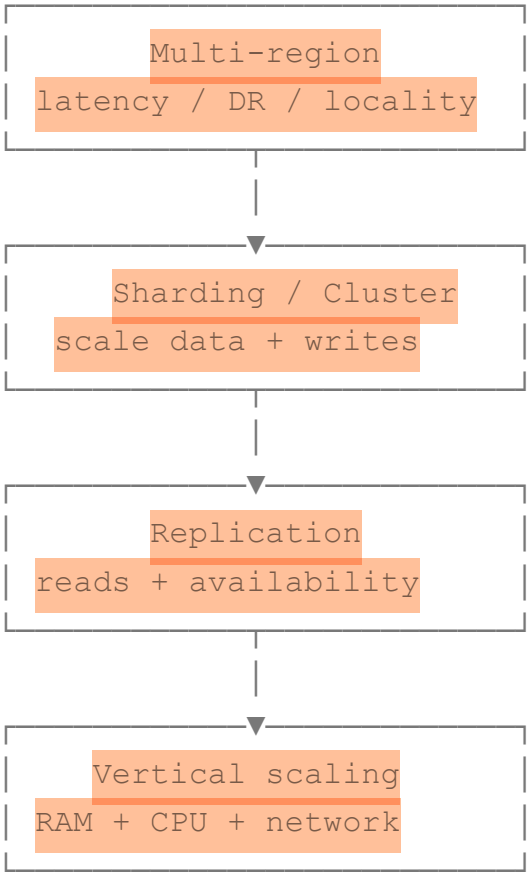
34. multi-AZ deployment
35. backup/recovery

36. cache stampede
37. thundering herd
38. cache warming
39. multi-region architecture
40. observability and capacity planning

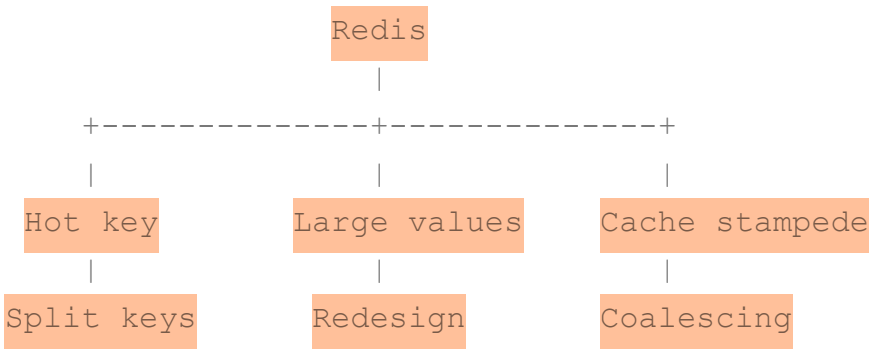
---

39. The mental model I recommend

For system-design interviews, remember Redis scaling as a **four-layer progression**:



And separately handle **workload pathologies**:



The most important architectural principle is:

> **Don't introduce Redis Cluster merely because traffic is high. First identify whether the bottleneck is memory, CPU, reads, writes, network, or a hot key. Use replication for read/availability scaling and partitioning for data/write scaling.**

For your interview preparation, the natural next topic is **Redis Cluster internals**—especially **hash-slot allocation, client-side routing, `MOVED` vs `ASK`, resharding, node failure/failover, and the exact scaling-up/scaling-down sequence**. That is where most of the deeper system-design questions come from.